

Relazione progetto PMCSN

Performance Modeling of Computer Systems and Networks - Anno accademico 2025/2026

ANDREA CANTARINI (0351377), Università degli Studi di Roma "Tor Vergata", IT

La presente relazione descrive il sistema oggetto di studio, gli obiettivi dello studio, la modellazione del sistema di simulazione, le modalità di verifica e validazione e i risultati ottenuti.

ACM Reference Format:

Andrea Cantarini (0351377). 2026. Relazione progetto PMCSN: Performance Modeling of Computer Systems and Networks - Anno accademico 2025/2026. 1, 1 (July 2026), 20 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Sistema oggetto di studio

Travis CI¹ è una piattaforma basata su cloud per la Continuous Integration e il Continuous Deployment (CI/CD), storicamente tra le più adottate per i progetti software open-source. Il suo scopo principale è automatizzare il testing e l'integrazione del codice: tramite un meccanismo di pipeline, la piattaforma si occupa di scaricare il codice, compilarlo ed eseguire i test, il tutto in maniera automatizzata e configurabile. Questo approccio garantisce che le nuove implementazioni non introducano difetti o conflitti nel software esistente.

La pipeline di Travis CI è definita in modo dichiarativo tramite un file di configurazione (`.travis.yml`) situato nella root del progetto. Quando viene innescato un evento (come un commit o la creazione di una pull request), Travis CI avvia una build su un *runner*.

Per ottimizzare i tempi e massimizzare la copertura dei test, una singola build viene tipicamente suddivisa in molteplici unità di lavoro atomiche chiamate Job. Spesso i progetti utilizzano una Build Matrix per lanciare diversi job in parallelo, permettendo di testare il medesimo codice simultaneamente su diverse versioni di un linguaggio di programmazione o diversi sistemi operativi.

Storicamente, per i progetti open-source, Travis metteva a disposizione 5 runner per repository. Tuttavia, a partire dal 2020, in seguito all'acquisizione da parte dell'azienda Idera Inc. e alla migrazione del dominio da `travis-ci.org` a `travis-ci.com`, il supporto gratuito per i progetti open-source è stato terminato². Queste modifiche hanno portato la gran parte dei progetti open-source a migrare verso altre piattaforme di CI/CD, prima tra tutte Github Actions.

Diverse tracce di elaborazioni su Travis CI sono state raccolte nel dataset TravisTorrent³, contenente i dati di circa quattro milioni di job eseguiti su quasi mille progetti open-source su Github tra il 2011 e il 2016.

¹<https://www.travis-ci.com/>

²<https://travis-ci.community/t/org-com-migration-unexpectedly-comes-with-a-plan-change-for-oss-what-exactly-is-the-new-deal/10567>

³<https://figshare.com/articles/dataset/TravisTorrent/19314170>

Author's Contact Information: Andrea Cantarini (0351377), andrea.cantarini@alumni.uniroma2.eu, Università degli Studi di Roma "Tor Vergata", Rome, Italy, IT.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

2 Obiettivi

Considerando il sistema descritto, il presente studio si pone quindi un duplice obiettivo:

- (1) analizzare le prestazioni di Travis CI come descritte nel piano gratuito per i progetti open-source.
- (2) stimare le prestazioni del sistema al variare del numero *runner* (*what-if analysis*).

3 Progettazione degli esperimenti

L'esperimento base consisterà nell'implementazione del modello così come descritto nelle prossime due sezioni della relazioni. In seguito, si varierà il numero di serventi nel sistema, eseguendo la simulazione con 1, 2, 5 e 10 *runner*, lasciando invariati gli altri parametri.

4 Modello concettuale

Per definire il modello concettuale del sistema, si è partito dall'analisi del dataset TravisTorrent.

Come prima cosa, è stata eseguita un'analisi sui dati presenti per i singoli progetti, il che ha portato alla selezione di uno solo tra questi: SonarSource/sonar-java⁴. Il progetto è stato determinato considerando la quantità di dati a disposizione e la bontà dei dati stessi. Inoltre, sono stati considerati i progetti le cui build presentano più job, così che l'accodamento si verifichi. Ad esempio, il progetto sonar-java presenta da 1 a 10 job per build.

La scelta di limitare lo studio a un solo progetto è stata dovuta al fatto che, tendenzialmente, progetti diversi presentano caratteristiche diverse, in primis tempi di interarrivo e di servizio. D'altra parte, ci si aspetta che i dati di un singolo progetto siano coerenti nel tempo.

4.1 Modellazione del ciclo di vita dei job

Come già descritto in precedenza, una build può portare alla creazione di più job. Una volta entrati in esecuzione, i job vengono sottoposti a due elaborazioni consecutive:

- *setup*, in cui viene predisposto l'ambiente di esecuzione, come definito nella build matrix
- *test*, in cui vengono eseguiti i test sul codice nell'ambiente predisposto.

Entrambe le fase sono eseguite in successione dallo stesso *runner*.

Al termine dell'esecuzione, un job può trovarsi in quattro stati diversi:

- *passed*: l'ambiente è stato predisposto correttamente e tutti i test sono andati a buon fine
- *failed*: l'ambiente è stato predisposto correttamente, ma almeno un test è fallito
- *errored*: si è verificato un errore durante la fase di setup, oppure è avvenuto un errore di sistema al termine dell'intera build
- *cancelled*: l'utente ha forzatamente interrotto l'esecuzione del job; l'interruzione può avvenire in qualsiasi momento dell'esecuzione.

Il diagramma di transizione degli stati dei job è descritto in Figura 1

Dato che i job vengono eseguiti sul codice, è possibile che lo stesso codice produca più elaborazioni. Infatti, nel dataset sono presenti job diversi lanciati a partire dallo stesso commit. È chiaro che, in questi casi, la mole di lavoro che i runner dovranno eseguire sarà identica per ogni esecuzione. Dal punto di vista concettuale, questo fenomeno viene modellato a seconda dello stato dei job.

⁴<https://github.com/SonarSource/sonar-java>

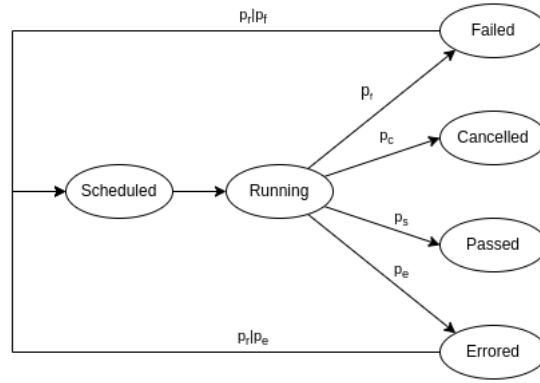


Fig. 1. Modello degli stati dei job

Un job nello stato *passed* non verrà più rieseguito. Nonostante il dataset mostri diversi job rieseguiti anche se passati, questi verranno considerati come nuovi ingressi nel sistema. Infatti, queste rielaborazioni sono principalmente dovuti a *cronjob* piuttosto che all'intervento umano. D'altra parte, un job può essere cancellato dal programmatore che si accorge di un errore nel codice dopo aver avviato la build e mentre questa è in esecuzione. Di conseguenza, il job in questione non sarà mai rieseguito, poiché il codice del progetto cambierà.

I job con stato *failed* o *errored* possono essere rieseguiti con una certa probabilità. Nello specifico, uno stato *errored* indica principalmente errori dovuto all'infrastruttura di Travis CI, come problemi di connettività, il che porta solitamente lo sviluppatore a rieseguire la build senza modifiche al codice. D'altra parte, lo stato *failed* richiede di norma modifiche al codice, ma non è detto che il programmatore se ne renda conto immediatamente, dunque potrebbe rieseguire la build con lo stesso codice. In generale, non si considera una rielaborazione certa in questi casi, dato che il codice sottostante potrebbe cambiare e, di conseguenza, anche il tempo di servizio dei job.

5 Modello delle specifiche

Il sistema viene modellato come un multiserver con 5 server, come mostrato in Figura 2.

Ogni server segue una distribuzione di Cox a due fasi, come mostrato in Figura 3:

- *setup*, che segue una distribuzione discreta la cui PMF è riportata in Tabella 1
- *test*, con distribuzione lognormale i cui parametri variano a seconda dello stato in cui il job finisce; i parametri sono riportati in Tabella 2 di parametri $\mu = 5.9545$ e $\sigma = 0.5316$

Le nuove richieste in entrata nel sistema seguono una distribuzione di Poisson di parametro $\lambda = 0.000045$. Nella pratica, questo si traduce con l'arrivo di una nuova richiesta ogni 6 ore circa.

Le probabilità che un job finisca in uno dei vari stati sono le seguenti:

- *passed*: 0.773436
- *failed*: 0.034525
- *errored*: 0.180305
- *cancelled*: 0.011734

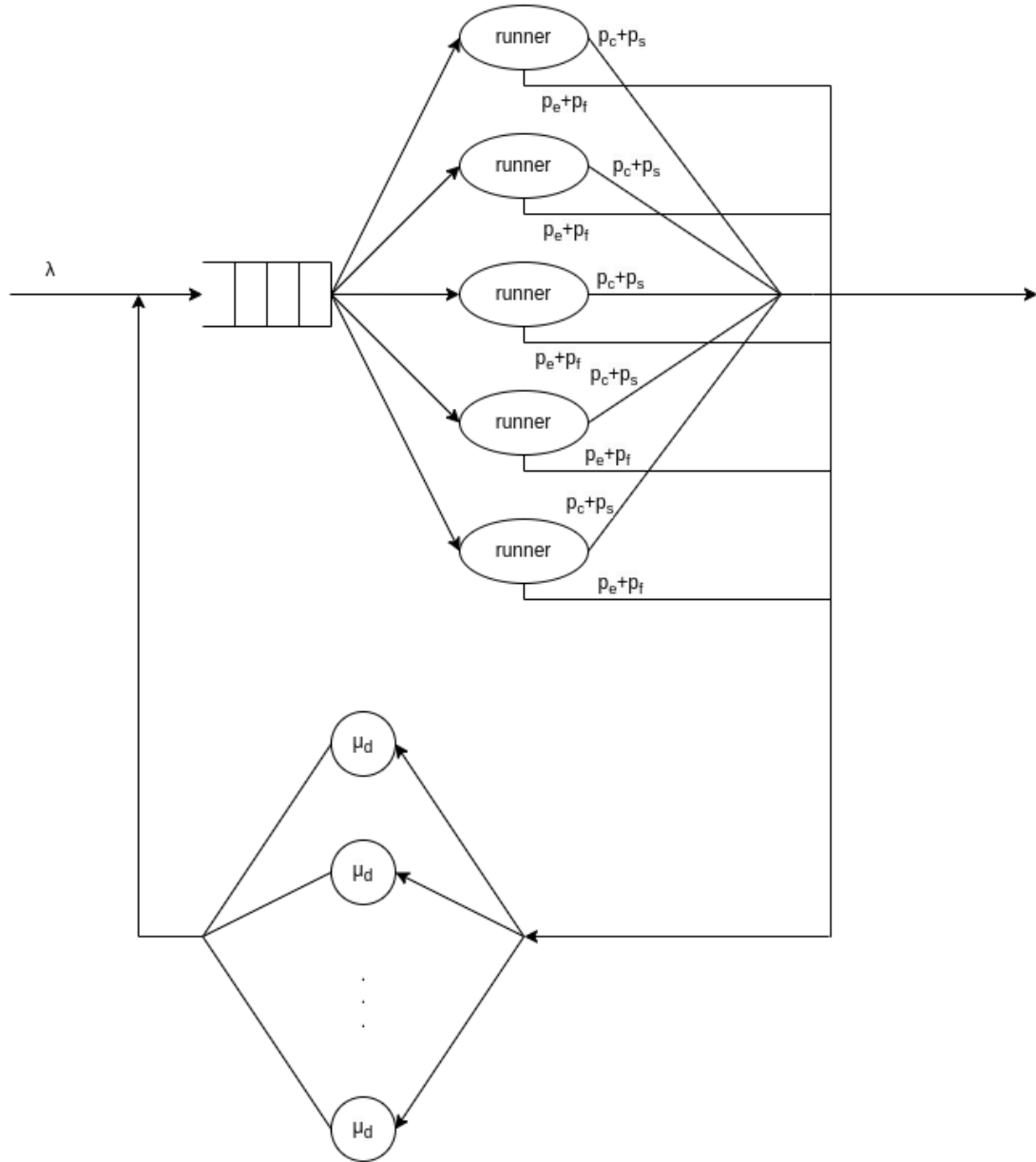


Fig. 2. Modello analitico

I job hanno una probabilità $p_{errored} = 0.8664$ di finire nello stato *errored* durante la fase di setup e $1 - p_{errored}$ di finirli al termine della fase di test.

Per quanto riguarda il ritardo nella rielaborazione dei job per via dell'intervento umano, questo è stato modellato come un sistema infinite-server in cui ogni servente segue una distribuzione lognormale di parametri $\mu = 0.9591$ e

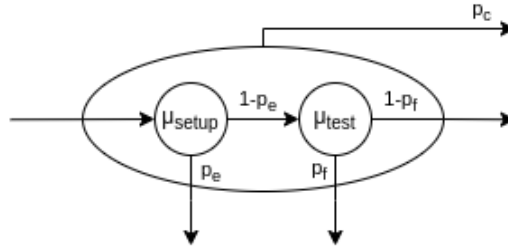


Fig. 3. Modello del runner

Tempo di servizio	Probabilità
1.0	0.491119
2.0	0.266563
3.0	0.115454
4.0	0.091534
5.0	0.012641
6.0	0.00739
7.0	0.003436
8.0	0.002204
9.0	0.00201
10.0	0.00201
11.0	0.001167
12.0	0.000648
13.0	0.000648
14.0	0.000324
15.0	0.000583
16.0	0.000324
17.0	0.000259
18.0	0.000259
19.0	0.000519
20.0	0.00013
21.0	0.000194
23.0	0.000194
28.0	0.000065
34.0	0.000065
46.0	0.000065
67.0	0.000065
76.0	0.000065
94.0	0.000065

Table 1. PMF della fase di setup dei job

$\sigma = 2.1399$. Il modello è mostrato in Figura 2. Le probabilità che i job in stato *failed* e *errored* vengano rielaborati - ossia, che i job rientrano del sistema passando per uno di questi server infiniti - sono, rispettivamente, 0.3327 e 0.8637.

Stato	μ	σ
<i>passed/errored</i>	5.9545	0.5316
<i>failed</i>	5.7644	0.5958
<i>canceled</i>	5.6105	0.6168

Table 2. Parametri della fase di test in funzione dello stato del job

6 Modello computazionale

Il modello computazionale è stato implementato in Python con un paradigma next-event. È stato utilizzato l'algoritmo di Welford per il calcolo delle medie e della varianza delle metriche raccolte. Come generatore random è stato utilizzato un generatore di Lehmer multi-stream.

Il codice sorgente del modello computazione è disponibile sulla repository del progetto⁵.

7 Verifica

La fase di verifica del modello computazione ha coinvolto nell'analisi dei dati relativi alle metriche raccolte. In particolare, è stato verificato che le metriche raccolte siano non-negative e che la Legge di Little sia rispettata.

8 Validazione

Il modello computazionale è stato valutato rispetto ai dati presenti nel dataset TravisTorrent per il progetto sonar-java. Si è verificata in particolare l'aderenza dei valori medi calcolati per tempi di servizio, tasso di arrivo, probabilità di feedback, probabilità che i job finiscano nei vari stati e probabilità di dimensionamento del batch di job in arrivo. I confronti sono riportati in Tabella 3.

Alcuni di questi confronti sono anche riportati nelle Figure 4, 5 e 6.

9 Analisi del transitorio

Per ognuno degli esperimenti è stata effettuata un'analisi del transitorio. Questa analisi è stata eseguita con una simulazione a orizzonte finito su cinque run diverse, utilizzando per ognuna un seed diverso. Le Figure 7, 8, 9 riportano i dati raccolti. Gli stessi dati sono consultabili in forma numerica nelle Tabelle 4, 5, 6. Tutti gli intervalli di confidenza sono stati calcolati su una distribuzione t-Student con 4 gradi di libertà.

10 Simulazione a orizzonte infinito

È stata eseguita una simulazione a orizzonte infinito con il metodo *batch means*, per analizzare la stazionarietà del sistema. Nello specifico, per ogni esperimento, è stata effettuata una simulazione con 64 batch da 500-1000 job ognuno. I risultati per tempo di servizio, tempo di risposta, tempo di attesa in coda, utilizzazione e popolazione in coda e nel sistema sono presentati graficamente nelle Figure 10, 11, 13, 12, 14 e 15 e numericamente nelle Tabelle 7, 8, 9, 10, 11 e 12, rispettivamente. Tutti gli intervalli di confidenza sono stati calcolati su una distribuzione t-Student con 63 gradi di libertà.

Infine, le Figure 16, 17 e 18 mostrano il variare, in media, del tempo di servizio, tempo di risposta e utilizzazione al variare del numero di serventi.

⁵<https://github.com/cantarell-light-12pt/ProgettoPMCSN>

Sezione	Voce	Reale	Simulato	Differenza
Outcome	<i>passed</i>	0.7734	0.7731	0.0004
Outcome	<i>failed</i>	0.0345	0.0362	0.0017
Outcome	<i>errored</i>	0.1803	0.1788	0.0015
Outcome	<i>canceled</i>	0.0117	0.0119	0.0002
Probabilità di feedback	<i>failed + errored</i>	0.2148	0.2150	0.0002
Tempo di servizio	<i>passed</i>	439.50s	446.01s	6.51s
Tempo di servizio	<i>failed</i>	382.17s	378.24s	3.93s
Tempo di servizio	<i>errored</i>	57.72s	59.27s	1.55s
Tempo di servizio	<i>canceled</i>	327.288s	325.46s	1.83s
Dimensione del batch	1	0.0426	0.0449	0.0023
Dimensione del batch	2	0.0189	0.0201	0.0012
Dimensione del batch	3	0.0457	0.0442	0.0014
Dimensione del batch	4	0.0755	0.0764	0.0009
Dimensione del batch	6	0.1058	0.1104	0.0046
Dimensione del batch	7	0.0781	0.0815	0.0034
Dimensione del batch	8	0.3468	0.3337	0.0131
Dimensione del batch	9	0.1962	0.1948	0.0014
Dimensione del batch	10	0.0904	0.0940	0.0036
Tasso di arrivo	build/s	4.675090×10^{-05}	4.636000×10^{-05}	0.0084

Table 3. Confronti tra i valori misurati sul modello computazionale e quelli ricavati dal dataset TravisTorrent

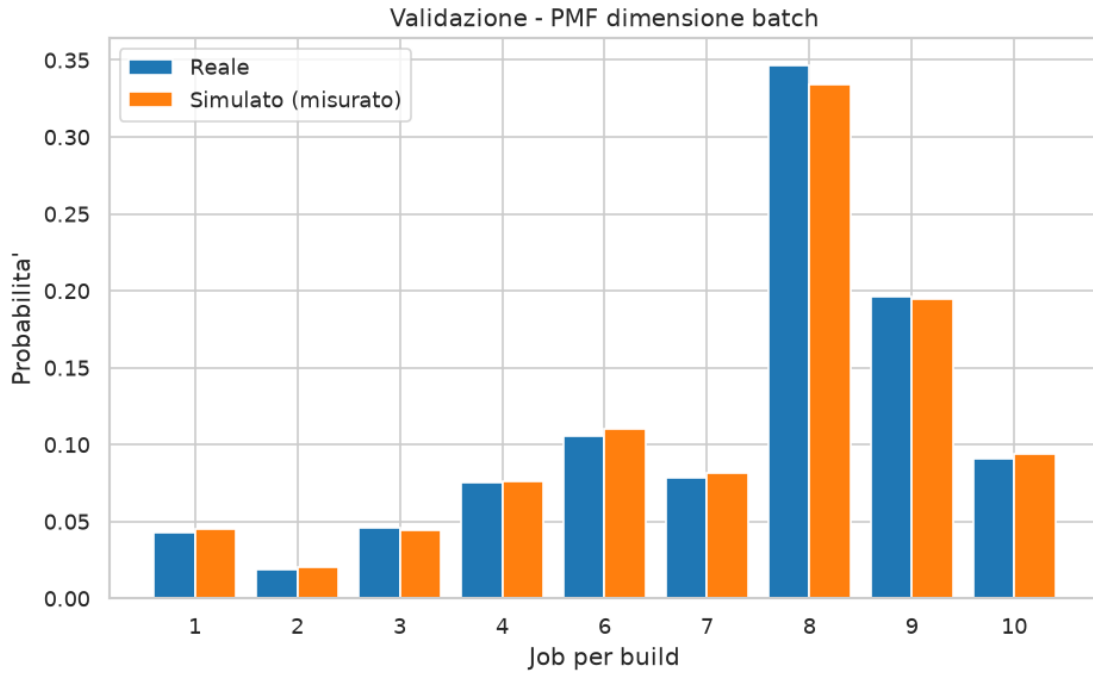


Fig. 4. Confronto tra le probabilità di dimensionamento dei batch reali e misurate

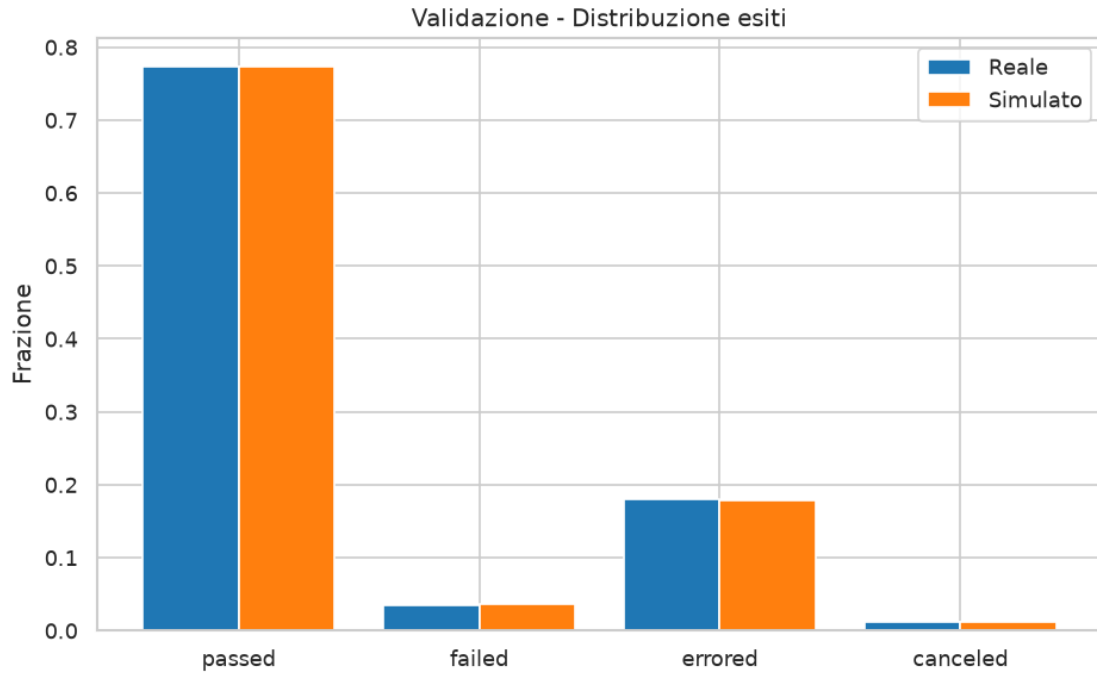


Fig. 5. Confronto tra le probabilità che un job finisca in uno dei quattro stati reali e misurate

Numero serventi	Media repliche	Intervallo di confidenza al 95%
1	446.877592	2.075169
2	446.858928	2.139395
5	446.817460	2.114599
10	446.912517	1.970302

Table 4. Risultati sul tempo di servizio dall'analisi del transitorio del variare del numero di serventi

Numero serventi	Media repliche	Intervallo di confidenza al 95%
1	2702.150589	158.095920
2	1562.804468	172.008574
5	1006.839952	179.159008
10	908.047567	179.150384

Table 5. Risultati sul tempo di risposta dall'analisi del transitorio del variare del numero di serventi

Numero serventi	Media repliche	Intervallo di confidenza al 95%
1	0.141900	0.011692
2	0.070550	0.006763
5	0.029860	0.006571
10	0.015250	0.003947

Table 6. Risultati sull'utilizzazione dall'analisi del transitorio del variare del numero di serventi

Numero serventi	Media	Intervallo di confidenza al 95%
1	445.683519	1.070444
2	445.685006	1.074078
5	445.694990	1.030969
10	445.688610	1.049126

Table 7. Media e IC del tempo di servizio al variare del numero di serventi (batch means)

Numero serventi	Media	Intervallo di confidenza al 95%
1	2662.834678	45.664897
2	1529.745584	44.359173
5	972.926641	44.172243
10	873.048490	44.365548

Table 8. Media e IC del tempo di risposta al variare del numero di serventi (batch means)

Numero serventi	Media	Intervallo di confidenza al 95%
1	1791.171579	14.127828
2	658.052273	4.705245
5	101.450373	0.983978
10	1.470376	0.169923

Table 9. Media e IC del tempo di attesa in coda al variare del numero di serventi (batch means)

Numero serventi	Media	Intervallo di confidenza al 95%
1	0.147723	0.001830
2	0.073271	0.001244
5	0.029578	0.000678
10	0.014734	0.000364

Table 10. Media e IC dell'utilizzazione al variare del numero di serventi (batch means)

Numero serventi	Media	Intervallo di confidenza al 95%
1	0.588368	0.009693
2	0.218968	0.006763
5	0.033849	0.001400
10	0.000357	0.000181

Table 11. Media e IC della popolazione in coda al variare del numero di serventi (batch means)

Numero serventi	Media	Intervallo di confidenza al 95%
1	0.736091	0.011160
2	0.365510	0.007062
5	0.181737	0.004456
10	0.147698	0.003673

Table 12. Media e IC della popolazione nel sistema al variare del numero di serventi (batch means)

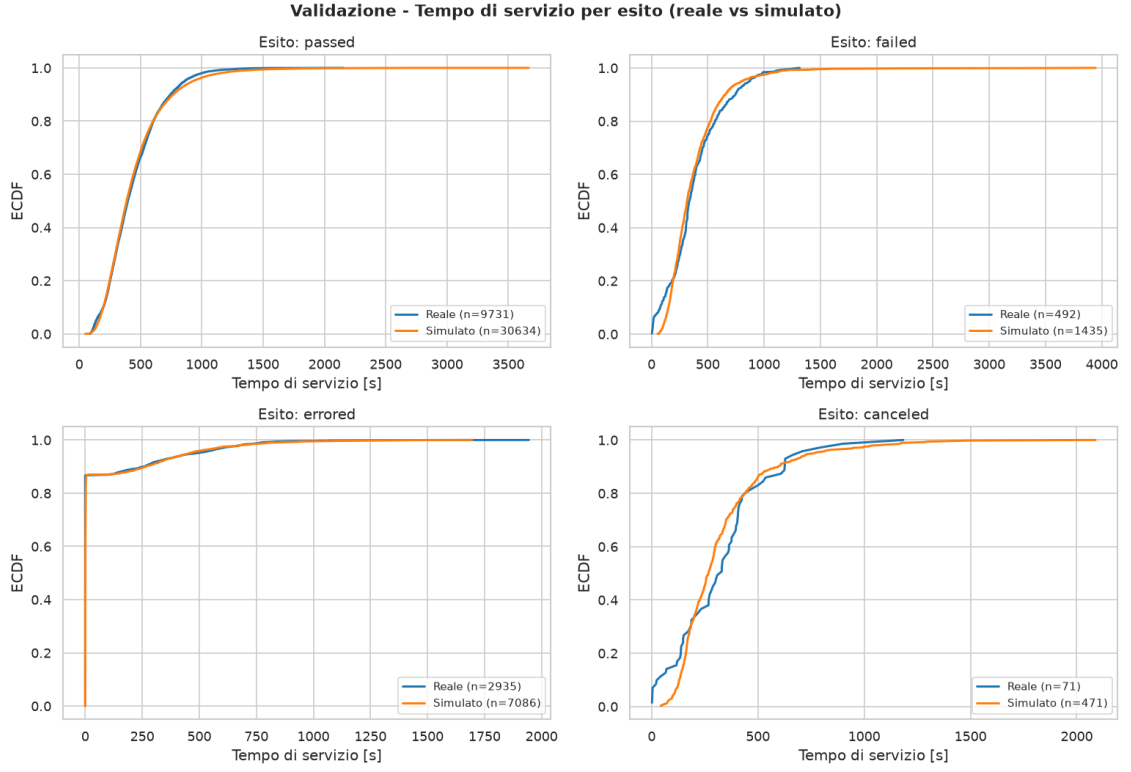


Fig. 6. Confronto tra le distribuzioni dei tempi di servizio per ognuno dei quattro stati reali e misurati

11 Conclusioni

Sono state quantificate le differenze di prestazioni del sistema in funzione del numero di serventi. Gli sviluppatori dei progetti open-source che hanno necessità di continuare a utilizzare Travis CI nonostante i costi, possono scegliere il piano per loro più appropriato a seconda dei requisiti imposti sui tempi di build.

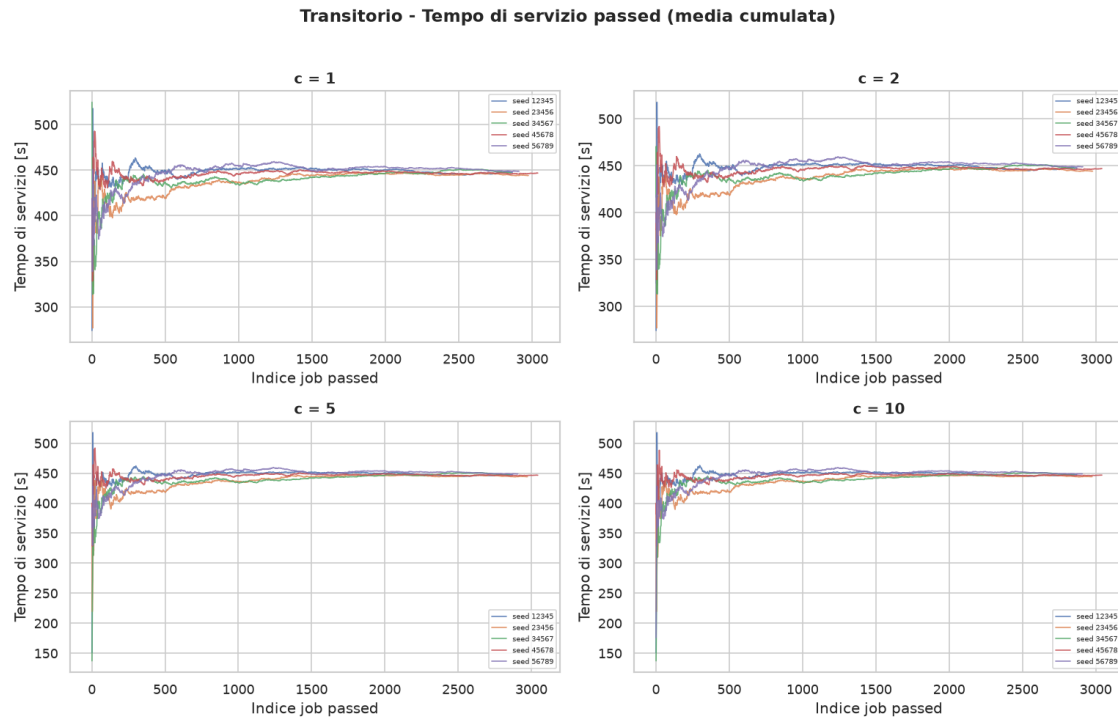


Fig. 7. Analisi del transitorio del tempo di servizio del sistema al variare del numero di serventi

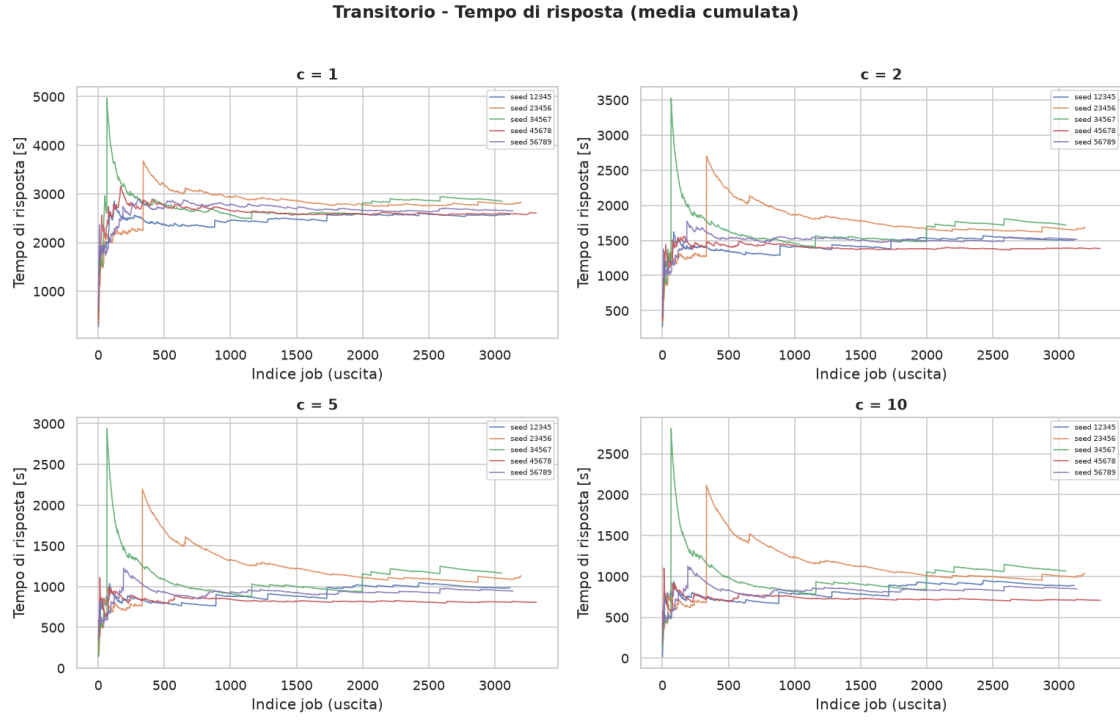


Fig. 8. Analisi del transitorio del tempo di risposta del sistema al variare del numero di serventi

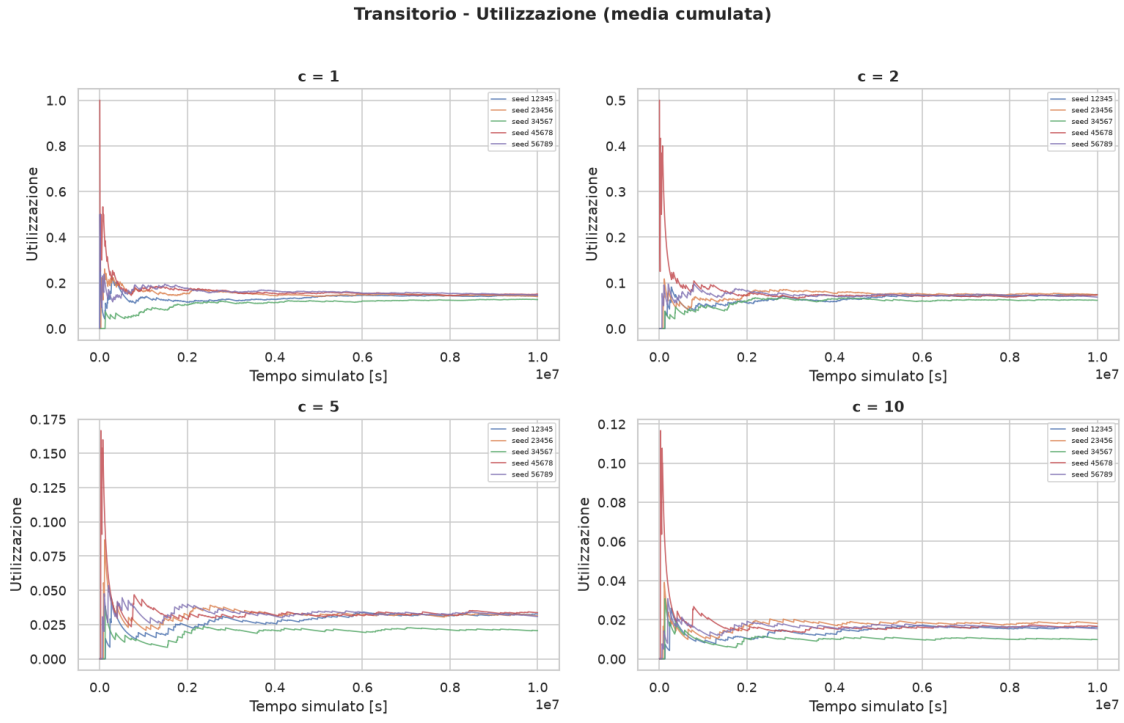


Fig. 9. Analisi del transitorio dell'utilizzazione dei server al variare del loro numero

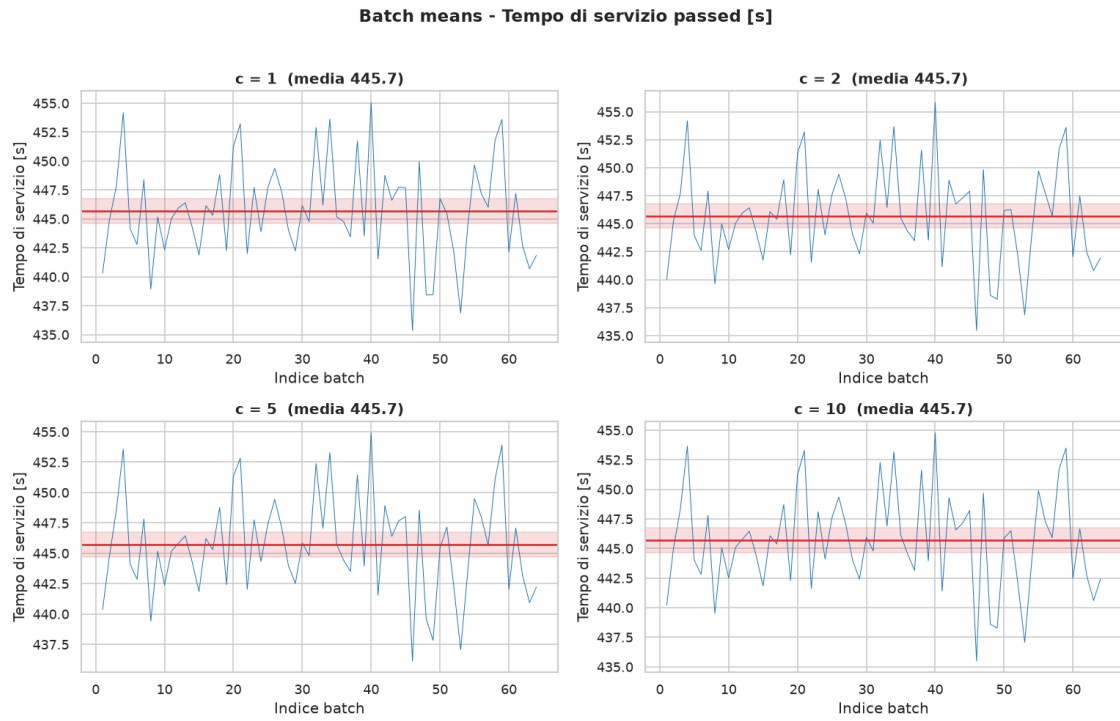


Fig. 10. Tempo di servizio (batch means)

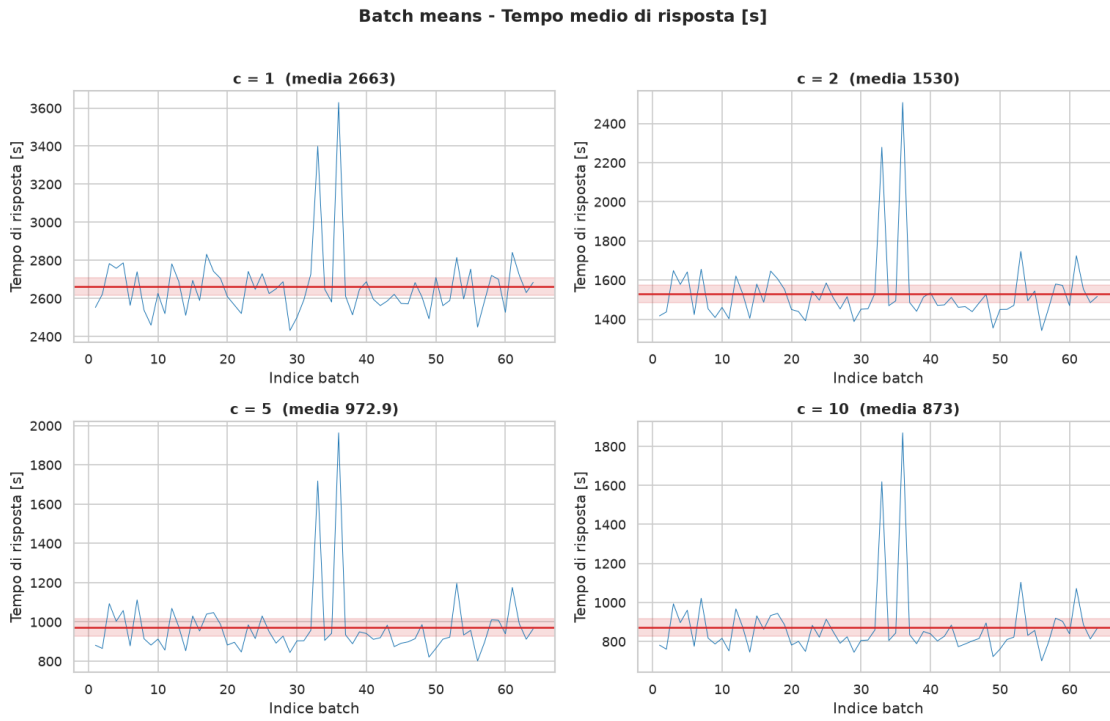


Fig. 11. Tempo di risposta (batch means)

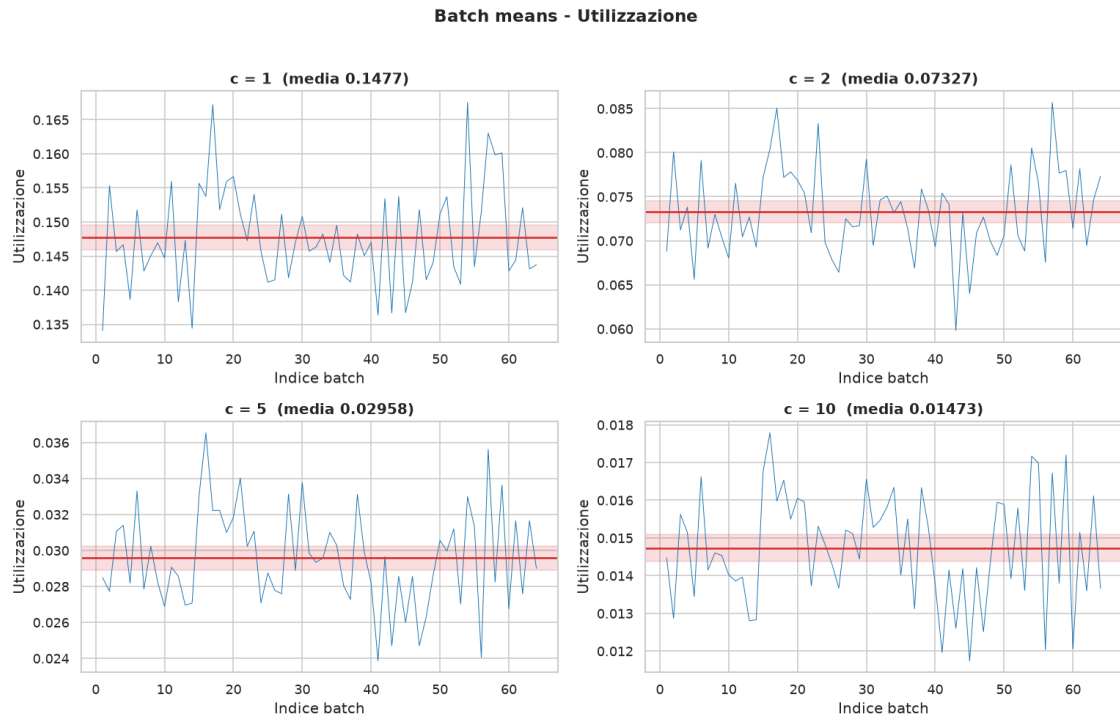


Fig. 12. Utilizzazione (batch means)

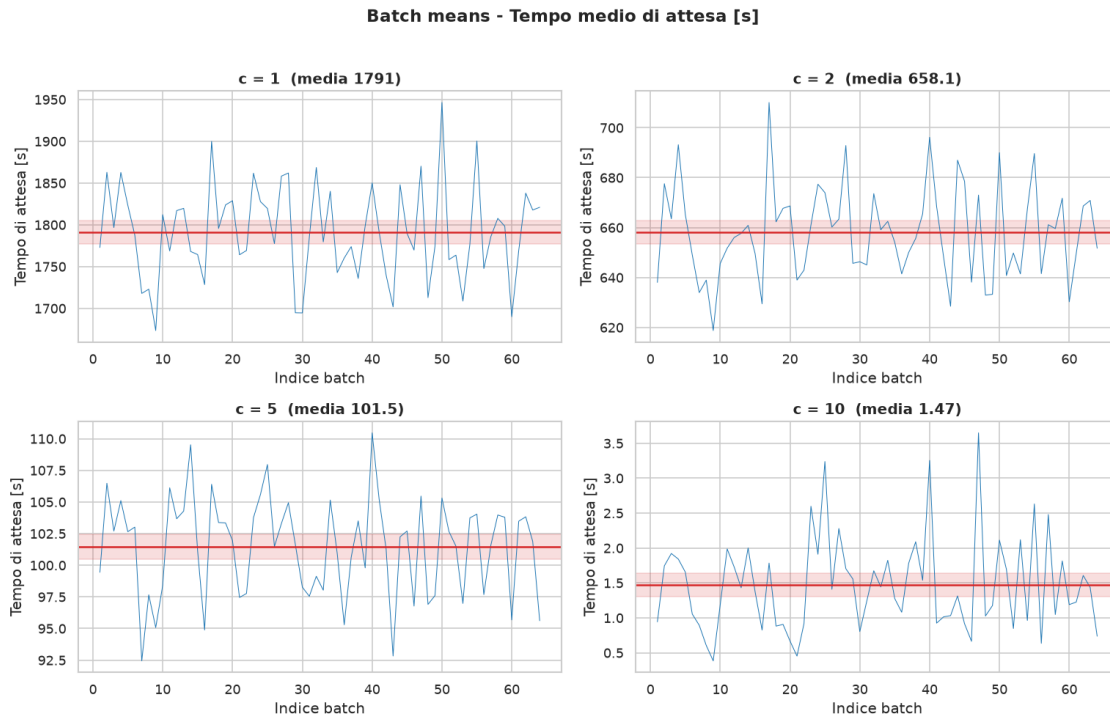


Fig. 13. Tempo di attesa in coda (batch means)

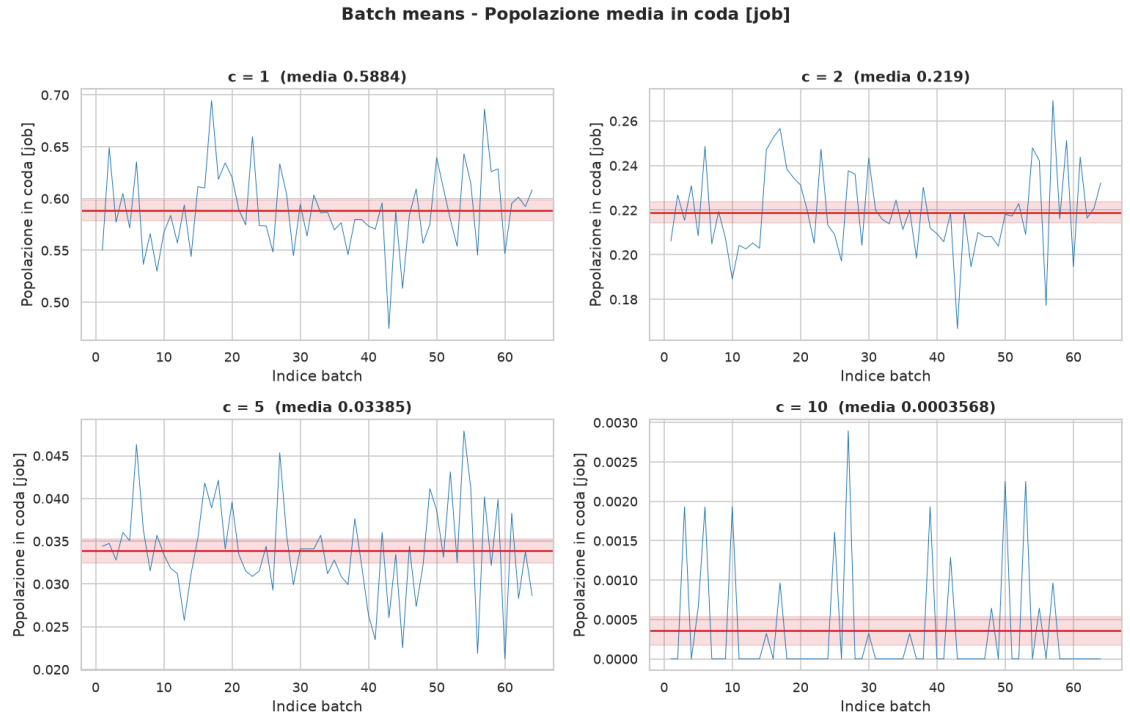


Fig. 14. Popolazione in coda (batch means)

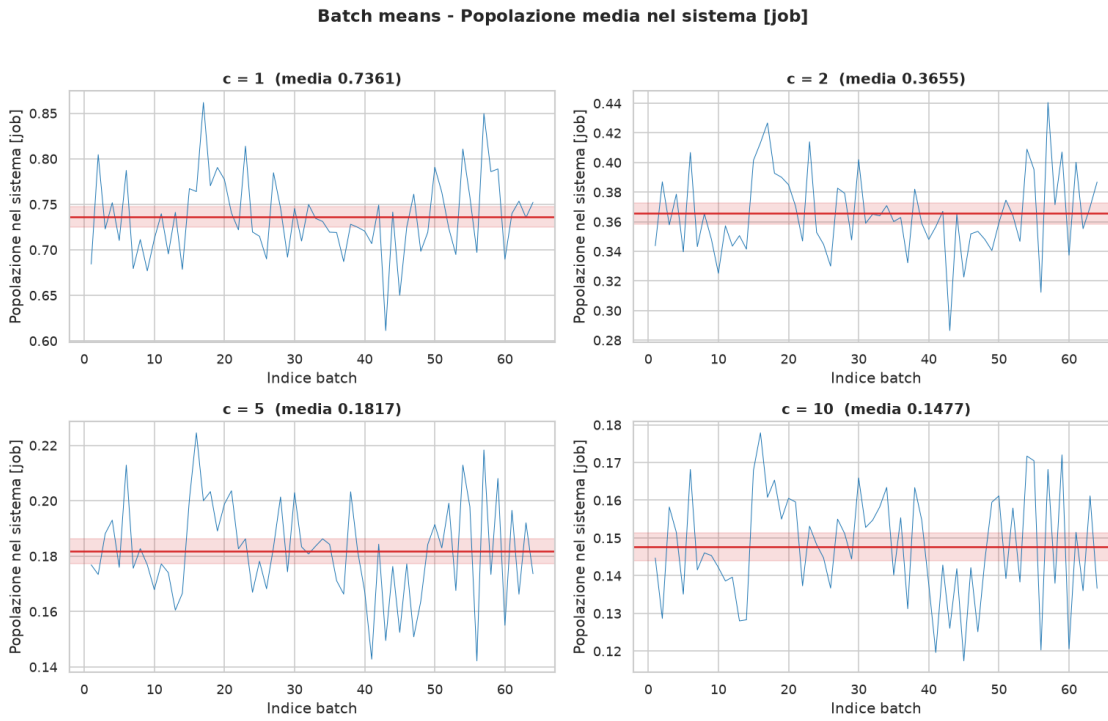


Fig. 15. Popolazione nel sistema (batch means)

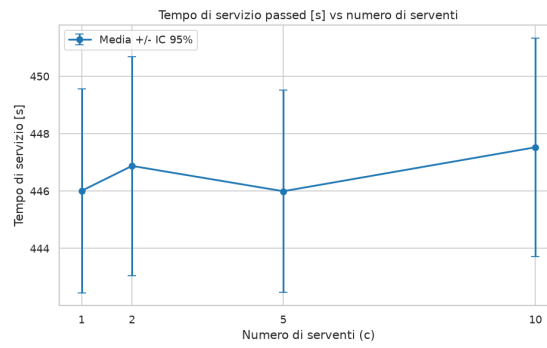


Fig. 16. Variazione del tempo di servizio al variare del numero di serventi

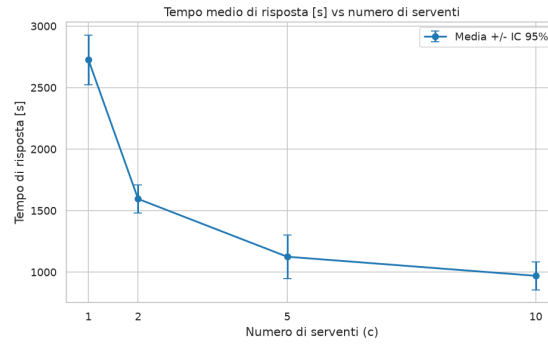


Fig. 17. Variazione del tempo di risposta al variare del numero di serveri

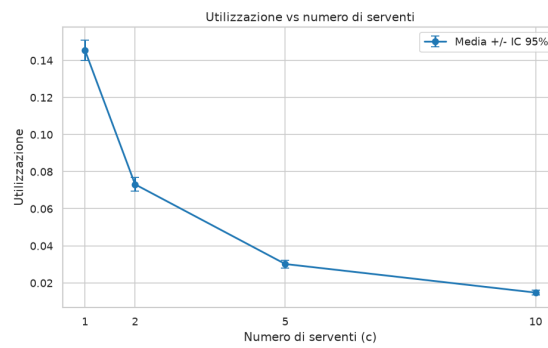


Fig. 18. Variazione dell'utilizzazione al variare del numero di serveri